535510 RL Team Project Final Report: Learning Curve-Informed Reinforcement Learning for Efficient Hyperparameter Optimization

Han-Syuan Liao* **Cheng-An Tsai*** **Yi-Xiang Yu***
Department of Computer Science
National Yang Ming Chiao Tung University
{hans.cs12, chengan81.ii14, seann.cs12}@nycu.edu.tw
* Equal contribution.

1 Project Overview

1.1 Profile

Track: 3. Application

Abstract and project goal: Hyperparameter optimization (HPO) is a fundamental yet costly challenge in machine learning: finding the right hyperparameter configuration often requires evaluating dozens of candidate configurations, each demanding a full or partial training run. Reinforcement learning (RL)-based HPO methods, such as Hyp-RL, have shown promise by learning a sequential decision policy over the hyperparameter space. However, existing RL-based approaches treat each configuration evaluation as a black-box query, discarding the rich temporal structure embedded in the learning curve produced during training.

In this project, we propose a learning curve-informed RL framework for HPO and explore two directions. (i) Derivative-informed state representation: We augment the agent's input at each step with explicit first- and second-order derivatives of the observed learning curve, providing structured inductive bias about whether a configuration is still improving, decelerating, or plateauing, at zero additional evaluation cost. (ii) Cross-dataset policy transfer: We investigate whether pre-training the LSTM policy across multiple datasets can yield an agent that generalizes to unseen datasets without restarting from scratch. Empirically, on the homogeneous LCBench benchmark the derivative features yield no measurable gain, whereas on the heterogeneous Kaggle Playground Series they give the curve-informed agent a consistent edge over its curve-free ablation—indicating that learning-curve derivatives help precisely when curve shapes vary across configurations.

TL;DR ("Too Long; Didn't Read"): We augment an RL-based HPO agent with learning curve derivatives as structured state features, and investigate cross-dataset transfer of the learned policy for more efficient and generalizable hyperparameter search.

1.2 Motivation

- **Why is the problem interesting?**
  When human experts tune hyperparameters, they rely on the geometry of the learning curve—observing how fast the loss is dropping or whether it is plateauing—rather than treating each evaluation as an isolated scalar. We aim to provide exactly this information to an RL agent and examine whether it improves tuning efficiency. Beyond single-task HPO, a practically useful agent should also transfer its experience to new datasets, exploiting the structural regularity that learning curves share across tasks.

- **Critical challenges.**
  Challenge 1: Reliable derivative estimation from noisy learning curves. Learning curves in practice are noisy, and finite-difference estimates can amplify this noise, potentially misleading the agent. Key design questions include window size, smoothing strategy, and sensitivity of the learned policy to these choices—all of which require empirical ablation.
  Challenge 2: Learning a transferable HPO policy across datasets. A practically useful HPO agent should generalize across tasks—given experience on a collection of datasets, it should be able to search effectively on a new, unseen dataset without retraining from scratch. This is non-trivial because different datasets induce different reward scales, convergence speeds, and hyperparameter sensitivities, making it difficult to learn representations that remain meaningful across tasks.

- **Justify why the problem remains open or unsolved.**
  We hypothesize that these derivative features provide a universal view of training dynamics. Since their physical meanings don't change between tasks, they make it easier for the RL agent to transfer its knowledge to new problems. Existing RL-based HPO works observe only scalar rewards, ignoring within-curve structure [Jomaa et al., 2019]. Learning curve-aware methods operate outside the RL framework [Wistuba et al., 2022]. Meta-RL approaches for HPO address cross-task generalization but rely on complex encoders without exploiting derivative structure [Albrechts et al., 2025, Wu et al., 2023]. Our work is the first to combine derivative-informed state representation with cross-dataset transfer in a unified RL-for-HPO framework.

- **State-of-the-art methods.**
  We consider Hyp-RL [Jomaa et al., 2019] as the state-of-the-art RL-based HPO baseline that our method directly builds upon.

2 Problem Formulation

In this work, we formulate the sequential hyperparameter optimization problem as a Markov Decision Process (MDP). Let Λ denote the H-dimensional hyperparameter search space. The MDP is defined by the tuple $\mathcal{M} \coloneqq (\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The action space is defined as $\mathcal{A} \coloneqq \Lambda$, where an action $a_t \in \mathcal{A}$ corresponds to selecting a specific hyperparameter configuration λ_t to evaluate.

The state space $\mathcal{S}$ is defined as $\mathcal{S} \coloneqq \mathbb{R}^w \times (\Lambda \times R \times \mathcal{C})^*$, where each state $s_t \in \mathcal{S}$ encapsulates both the static dataset meta-features $\mathbf{d} \in \mathbb{R}^w$ and the dynamic history of previously evaluated configurations alongside their corresponding outcomes, including $\mathcal{C} \subset \mathbb{R}^K$ being the space of learning curves, representing a sequence of validation losses over K training epochs and containing the information of the first and second derivatives of the learning curve.

The reward function is defined as $R(D, a) = -f(D, \lambda = a)$ on a given dataset D. The deterministic transition function P updates the state by appending the newly selected configuration and observed reward to the historical sequence, yielding $s_{t+1} = (s_t, (a_t, r_t, c_t))$. Furthermore, we impose strict termination constraints on the environment: an episode concludes either when a predefined budget sequence length T is exhausted, or if the agent selects the identical action consecutively ($a_t = a_{t-1}$).

Our objective is to learn an optimal policy $\pi^* : \mathcal{S} \to \mathcal{A}$ that maximizes the expected discounted cumulative reward, effectively identifying the hyperparameter configuration that minimizes the generalization error across varying datasets. This is achieved by estimating the optimal action-value function $Q^*(s, a)$, which satisfies the Bellman equation:

$$Q^*(s, a) = \mathbb{E}_\pi \left[r + \gamma \max_{a'} Q^*(s', a') | S_0 = s, A_0 = a \right]$$

where $\gamma \in [0, 1]$ is the discount factor. To operate within this high-dimensional state space, which consists of variable-length temporal sequences and distinct response surfaces, we employ a parameterized function approximator $Q(s, a; \theta)$. Specifically, we utilize a Long Short-Term Memory (LSTM) network to model the dynamic action-value space, enabling the agent to capture long-range dependencies and transfer optimization knowledge across datasets.

3 Empirical Evaluation

3.1 Performance Metrics

We plan to evaluate the algorithms based on the following four metrics:

- Final validation performance: We record the best validation loss obtained by the model trained with the hyperparameters discovered by the agent, using the task-appropriate metric (e.g. RMSE for regression, log-loss for classification).
- Search efficiency: This metric evaluates how quickly the agent converges to strong configurations. We quantify it by the number of function evaluations (the evaluation budget), reporting performance as a function of the budget so that cold-start and asymptotic behavior can be read off a single anytime curve.
- Simple regret: Derived from the multi-armed bandit literature, simple regret measures the loss gap between the best configuration found by any compared method (the oracle) and the agent's recommended configuration. Because the benchmark spans widely varying loss scales, we report *normalized* simple regret to make the metric comparable across tasks.

3.2 Baseline Methods

We compare against the following standard HPO baselines:

- Random Search [Bergstra and Bengio, 2012]: A fundamental baseline that randomly samples hyperparameter configurations. It serves as a necessary lower bound to verify that the proposed RL agent explores high-dimensional continuous search spaces more effectively than random selection.
- BO [Snoek et al., 2012]: A standard state-of-the-art method for HPO. We will use a Gaussian Process-based BO with Expected Improvement (EI) to systematically benchmark against our RL agent's sequential decision-making capabilities.
- Hyp-rl [Jomaa et al., 2019]: An RL-based HPO method that learns a sequential decision policy via DQN over a discretized hyperparameter space, using only scalar reward feedback without learning curve information.

In addition, we evaluate two variants of our proposed method:

- **CrossDataset-HyperRL**: A Hyp-RL agent pre-trained across multiple source datasets and evaluated on held-out target datasets, isolating the contribution of cross-dataset policy transfer (idea ii).
- **CrossDataset-LC-DQN**: Our full method, which combines cross-dataset pre-training with the derivative-informed state representation (idea i), augmenting the agent's input with first- and second-order derivatives of the observed learning curves.

3.3 Benchmark Tasks or Datasets

We evaluate the algorithms in various benchmark domains widely used in the literature, specifically focusing on continuous prediction tasks:

- **LCBench** [Zimmer et al., 2021]: A learning curve benchmark built on top of OpenML datasets, providing pre-computed training trajectories of funnel-shaped MLPs across 35 tabular classification tasks. Each of its 2,000 hyperparameter configurations per dataset is logged with per-epoch validation losses over 50 epochs, exposing the full learning curve required by our derivative-informed state representation. The shared 7-dimensional search space (batch size, learning rate, momentum, weight decay, number of layers, max units per layer, dropout) and aligned training protocol across datasets make LCBench particularly well-suited for evaluating cross-dataset policy transfer.
- **Kaggle Playground Series** [Kaggle, 2023–2025]: To evaluate the generalization and cross-dataset transfer capabilities of the reinforcement learning policy, we utilize 15 tabular datasets sourced from the Kaggle Playground Series competitions. This benchmark suite

encompasses variations in feature dimensions, instance counts, and mathematical task types (spanning continuous regression and discrete classification). The datasets, identified by their competition season and episode (e.g., `playground-series-s3e1`), consist of synthetic data generated via deep learning models trained on real-world reference data. This generation mechanism ensures the preservation of underlying statistical distributions and feature correlations, providing a standardized, heterogeneous environment for evaluating policy transferability.

4 Methodology

Our framework, illustrated in Figure 1 and Algorithm 1, extends the Hyp-RL formulation [Jomaa et al., 2019] along two axes: a derivative-informed state representation and a cross-dataset meta-training procedure. Both share the same LSTM policy and differ only in state features and training protocol.

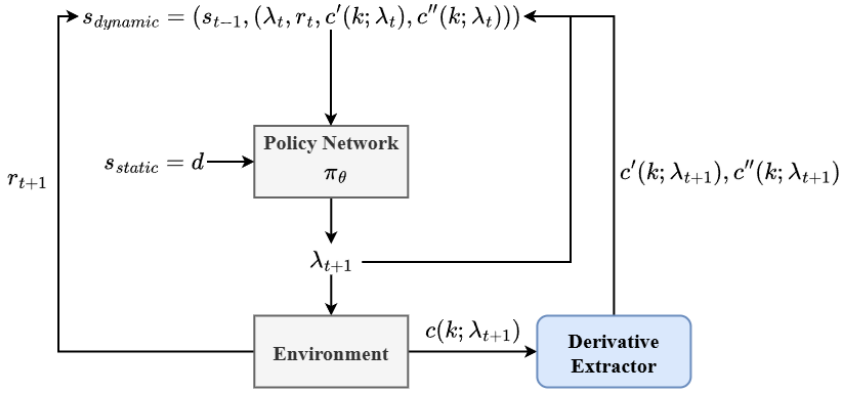


Figure 1: Framework

Algorithm 1 CrossDataset-LC-DQN Meta-Training

Require: Source datasets $\mathcal{D}$, Total episodes E, Training budget T
Initialize shared policy network $Q(s, a; \theta)$ and replay buffer $\mathcal{B}$
for episode $= 1$ **to** E **do**
  Sample a source dataset $D \sim \mathcal{D}$
  Extract static metadata descriptor d from D
  Initialize LSTM hidden and cell states (h_0, c_0) using d
  Initialize state sequence s_0 with d
  for $t = 1$ **to** T **do**
    Select hyperparameter configuration a_t via ϵ-greedy policy
    Evaluate a_t on D and obtain learning curve $c(k; a_t)$
    Extract curve features s_{lc} (including derivative statistics) from $c(k; a_t)$
    Compute improvement reward r_t
    Update dynamic state $s_t = (s_{t-1}, (a_t, r_t, s_{lc}))$
    Store transition (s_{t-1}, a_t, r_t, s_t) in $\mathcal{B}$
    Update network parameters θ by sampling mini-batches from $\mathcal{B}$
    if termination condition is met **then**
      break
    end if
  end for
end for

Derivative-informed state. Rather than feeding the full derivative sequence $c'(k)$, $c''(k)$ into the network, we summarize each observed learning curve with five compact finite-difference statistics over its last 8 epochs: the final value, minimum, mean first difference (slope), mean second difference

(curvature), and total improvement. These features are appended to each history row, increasing the input dimension by five. We refer to this variant as **CrossDataset-LC-DQN**, with **CrossDataset-HyperRL** denoting the ablation without curve features.

Dataset meta-features. We characterize each dataset using a 16-dimensional static descriptor $d \in \mathbb{R}^{16}$. This descriptor comprises structural and statistical meta-features extracted from the dataset, as detailed in Table 1. These 16 values are concatenated and L_2-normalized per dataset to form the static descriptor d. This descriptor is appended to every history row so that each per-step input is aware of the dataset's structural properties. Additionally, it is projected through two linear layers to initialize the LSTM hidden and cell states (h_0, c_0), priming the recurrent policy before any configuration is evaluated.

Table 1: Extracted Dataset Meta-Features

Number of Instances	Kurtosis Min
Log Number of Instances	Kurtosis Max
Number of Features	Kurtosis Mean
Log Number of Features	Kurtosis Standard Deviation
Data Set Dimensionality	Skewness Min
Log Data Set Dimensionality	Skewness Max
Inverse Data Set Dimensionality	Skewness Mean
Log Inverse Data Set Dimensionality	Skewness Standard Deviation

Cross-dataset meta-training. We learn a single shared LSTM across source datasets via round-robin sampling: each meta-training episode samples one source dataset, runs a full HPO episode of length T, and updates the shared policy from a shared replay buffer. To stabilize learning across heterogeneous loss scales, we use an improvement reward $r_t = \max(0, -f_t + f_{t-1})$. After meta-training, the policy is frozen ($\varepsilon = 0.01$) and evaluated on held-out target datasets, while non-meta baselines are reinitialized per target under the same budget.

5 Experimental Results

5.1 Experimental Setup

All RL agents share a single DQN configuration; the two variants differ only in whether the learning-curve derivative features (Section 4) are appended to the state. The Q-network is a single-layer LSTM with hidden size 64: the 16-dimensional, L_2-normalized dataset descriptor is projected by two linear layers to initialize the recurrent hidden and cell states (h_0, c_0), and the final hidden state is mapped to per-action Q-values by a two-layer MLP head ($64 \to 64 \to |\mathcal{A}|$ with ReLU). At each step the state is a zero-padded sequence of history rows, where each row concatenates the encoded configuration, the scalar improvement reward, the five derivative statistics (`LC-DQN` only), and the static descriptor. The action space $\mathcal{A}$ is the benchmark's precomputed configuration grid (sampled without replacement within an episode), and the reward is the validation-loss improvement $r_t = \max(0,\ f_{t-1} - f_t)$. Training follows standard DQN with a hard-updated target network, Bellman target $r + \gamma \max_{a'} Q_{\text{target}}(s', a')(1 - \text{done})$, and an MSE temporal-difference loss optimized with Adam. The shared hyperparameters are listed in Table 2.

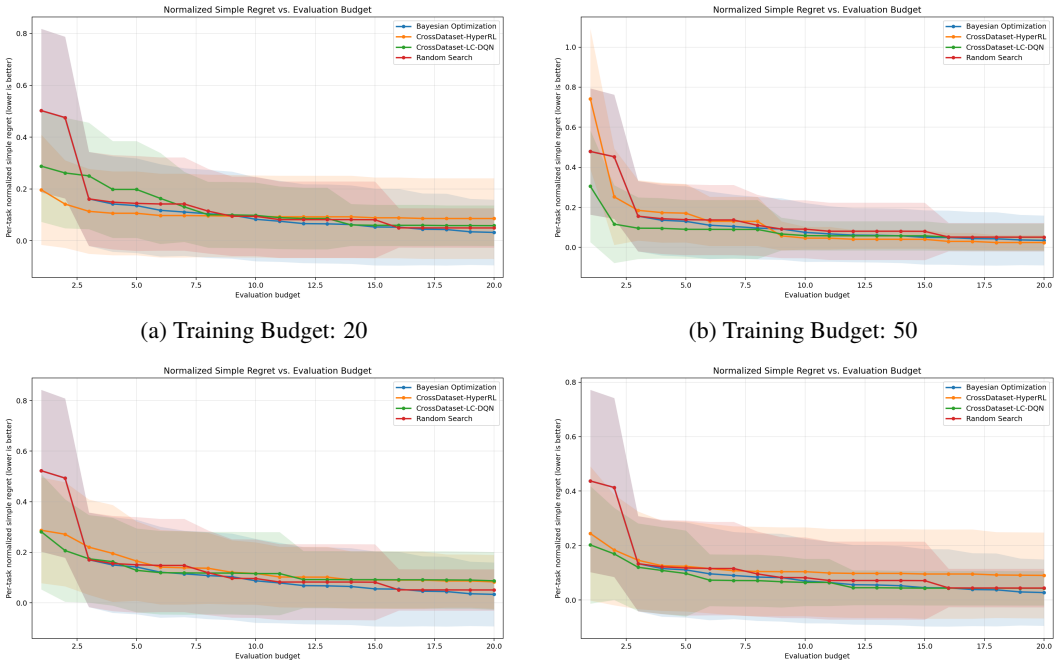
The two benchmarks differ only in their task collections and evaluation protocol. On **LCBench** we meta-train across all 35 OpenML datasets (shared 7-dimensional search space, up to 2,000 logged configurations per dataset with 50-epoch curves) using a single seed, and sweep the training budget $T \in \{20, 50, 100, 150\}$ while holding the per-task evaluation budget fixed at 20 (Figure 2). On the **Kaggle Playground Series** we use 15 heterogeneous tasks (256 cached configurations per task with 25-epoch curves), 5 random seeds, and a single training budget $T = 20$. Across both benchmarks the non-meta baselines (Random Search, Bayesian Optimization) are reinitialized per target task under the same evaluation budget, and the meta-trained policies are frozen after meta-training before evaluation.

Table 2: DQN agent and meta-training hyperparameters. E denotes the number of meta-training episodes and T the per-episode training budget (HPO rollout length).

Hyperparameter	Value
Q-network	1-layer LSTM (hidden 64) + MLP head ($64 \to 64 \to \lvert\mathcal{A}\rvert$)
Optimizer	Adam
Learning rate	1×10^{-3}
Discount factor γ	0.99
Replay buffer size	$\max(10^4,\ E \cdot T)$
Batch size	4
Learning starts	2 transitions
Target-network update	hard copy every 4 steps
Exploration ε (meta-training)	linear $1.0 \to 0.05$ over E episodes
Exploration ε (frozen evaluation)	0.01
Meta-training episodes E	150
Training budget T	LCBench: $\{20, 50, 100, 150\}$; Kaggle: 20
Evaluation budget	20

5.2 Evaluation of Baseline Methods

Based on the experimental results evaluating validation scores against evaluation budgets, the following technical observations are made regarding the performance of the optimization algorithms:



(a) Training Budget: 20

(b) Training Budget: 50

(c) Training Budget: 100

(d) Training Budget: 150

Figure 2: Performance comparison of hyperparameter optimization methods across different training budget constraints. Lower validation scores indicate better performance.

- **Initial Optimization Phase (Cold-Start):** At low evaluation budgets (e.g., trials 1 through 5), the `CrossDataset-HyperRL` and `CrossDataset-LC-DQN` methods yield lower validation scores compared to Bayesian Optimization and Random Search. This indicates a higher search efficiency and a more effective initialization during the early phase of the optimization process.

- **Meta-Learning and Knowledge Transfer:** This early-stage performance difference correlates with the underlying initialization mechanisms. `CrossDataset-HyperRL` utilizes metadata to transfer learned hyperparameter distributions from prior datasets to the current validation dataset. Conversely, standard Bayesian Optimization initiates the search process on each new dataset without prior external data, requiring more exploratory trials to model the objective function from scratch.
- **Asymptotic Convergence:** As the evaluation budget increases (beyond 15 to 20 trials), the validation scores of all evaluated methods converge to a similar numerical range. The data shows that given a sufficient evaluation budget, the surrogate model constructed by Bayesian Optimization achieves an equivalent performance level to the transferred policies of the meta-learning approaches.

5.3 Cross-Dataset Transfer on the Kaggle Playground Series

On LCBench, `CrossDataset-LC-DQN` and `CrossDataset-HyperRL` are essentially indistinguishable: at the smallest budgets the curve-free `HyperRL` ablation is even marginally ahead of the full method (e.g. the orange curve below the green one in Figure 2a). LCBench therefore cannot isolate the contribution of the derivative-informed state. To probe this, we meta-train the same two cross-dataset policies (`CrossDataset-LC-DQN` and `CrossDataset-HyperRL`) on the Kaggle source tasks and evaluate them with frozen weights on the 15 heterogeneous Kaggle Playground tasks, reporting per-task normalized simple regret against the evaluation budget (Figure 3); normalization makes regret comparable across the widely varying loss scales of the suite.

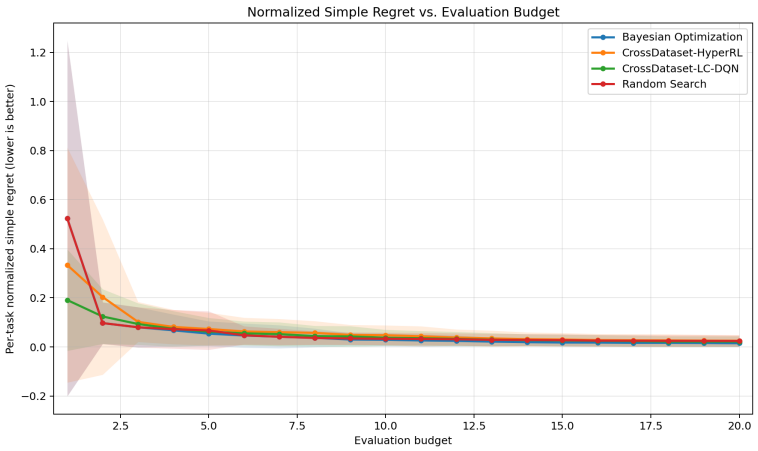


Figure 3: Normalized simple regret versus evaluation budget on the 15 Kaggle Playground tasks (5 seeds per task; shaded bands denote $\pm$ one standard deviation). Lower is better. Unlike LCBench, the curve-informed `CrossDataset-LC-DQN` (green) clearly separates from the curve-free `CrossDataset-HyperRL` (orange) throughout the cold-start region.

- **Derivative features isolate a measurable gain.** Across the low-budget region (trials 1–7), `CrossDataset-LC-DQN` sits consistently below `CrossDataset-HyperRL`. Averaged over all 15 tasks, its normalized simple regret is 0.0217 versus 0.0251 for `HyperRL`, and it also wins more tasks (rank-1 on 4 tasks vs. 2, average per-task rank 2.33 vs. 2.87). Because the two variants share the same policy, training protocol, and search space and differ *only* in the appended learning-curve derivative statistics, this gap directly attributes the improvement to the derivative-informed state.
- **Curve features are what beat the random baseline.** The curve-free `HyperRL` (0.0251) is in fact slightly *worse* than Random Search (0.0243) on average, whereas adding the derivative features pushes `LC-DQN` (0.0217) clearly below it. On this heterogeneous suite, cross-dataset policy transfer alone is not enough to outperform random sampling—the learning-curve signal is the deciding factor.
- **Bayesian Optimization remains strongest asymptotically.** As on LCBench, the GP surrogate attains the best overall normalized simple regret (0.0150) and dominates once the

budget is large. The advantage of the transferred RL policies is concentrated in the early cold-start regime, where a learned prior matters more than an on-the-fly surrogate.

Why curve features help on Kaggle but not on LCBench. LCBench is deliberately homogeneous: every task trains the same funnel-shaped MLP family over the same 7-dimensional search space for a fixed 50 epochs, so the logged validation curves are smooth, stereotyped, and largely monotone. A configuration's final value is then well predicted by the static dataset descriptor and the scalar reward already present in the `HyperRL` state, leaving the derivative statistics (slope, curvature, recent improvement) with little additional signal—and at times adding input noise, which explains why `LC-DQN` fails to improve over `HyperRL` there. The Kaggle suite, by contrast, spans both regression and classification over real-world tabular datasets with very different scales and training dynamics, so different configurations produce genuinely diverse curve geometries (steady improvers, early plateaus, overfitters). In this setting the derivative features supply discriminative cold-start information that the scalar-only state lacks, letting the curve-informed policy commit to promising configurations sooner. In short, the derivative-informed state pays off precisely when learning-curve *shape* varies across configurations—a property the heterogeneous Kaggle tasks exhibit and the uniform LCBench protocol largely removes.

Effect of meta-training task count. Figure 4 varies the number of source datasets used to meta-train `LC-DQN` $(1, 5, 10, 15)$ and reports normalized simple regret on the held-out targets (75 runs per setting). Transfer does not improve monotonically with more source tasks: regret is lowest at 15 tasks (0.0217) but the curve is non-monotonic, peaking at 10. We read this as a consequence of the suite's heterogeneity—adding dissimilar source datasets injects conflicting reward scales and curve dynamics that a single shared LSTM must reconcile, so additional tasks help only once enough are present to average out the mismatch. The standard-error bars overlap substantially, however, so this trend is weak and should be treated as suggestive rather than conclusive.

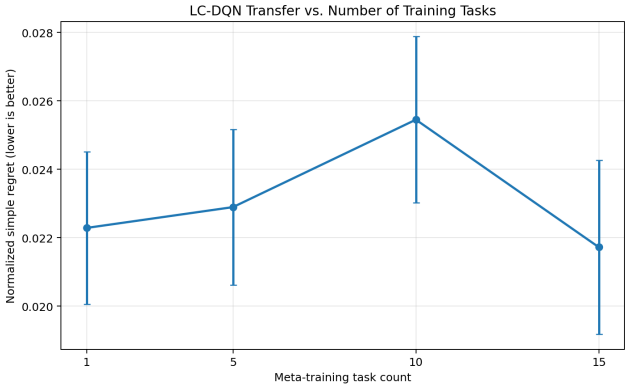


Figure 4: `CrossDataset-LC-DQN` transfer performance as a function of the number of meta-training (source) datasets. Error bars denote one standard error over 75 runs.

6 Conclusion

We extended the Hyp-RL formulation along two axes—a derivative-informed state representation and cross-dataset meta-training—and evaluated both on a homogeneous learning-curve benchmark (LCBench) and a heterogeneous suite of real-world tabular tasks (Kaggle Playground Series). Our central finding is that the value of the learning-curve derivative features is benchmark-dependent. On LCBench, where every task shares the same network family, search space, and training protocol, the curve-informed `CrossDataset-LC-DQN` is statistically indistinguishable from its curve-free ablation `CrossDataset-HyperRL`; the stereotyped, near-monotone curves leave little signal for the derivatives to exploit. On the heterogeneous Kaggle suite, by contrast, the same features produce a consistent cold-start advantage: `LC-DQN` attains lower average normalized simple regret than `HyperRL` $(0.0217$ vs. $0.0251)$ and is the only RL variant to cross the Random Search baseline. Because the two variants differ *only* in the appended derivative statistics, this gap directly attributes the improvement to the learning-curve information.

Two secondary observations qualify the result. First, Bayesian Optimization remains the strongest method asymptotically on both benchmarks; the RL agents' advantage is concentrated in the low-budget cold-start regime, where a transferred prior matters more than an on-the-fly surrogate. Second, cross-dataset transfer does not improve monotonically with the number of meta-training tasks, suggesting that naïvely pooling dissimilar datasets can inject conflicting reward scales and curve dynamics. Overall, learning-curve derivatives are a cheap, zero-evaluation-cost state augmentation that pays off precisely when curve shape varies across configurations—a property of practical, heterogeneous HPO workloads. This motivates the surrogate-accelerated extension discussed next, which would let the agent exploit this signal across far more datasets without the cost of full training runs.

7 Expected Contributions of Each Team Member

- Han-Syuan Liao: Focuses on result presentation, including data visualization, poster design, and delivering the final presentation.
- Cheng-An Tsai: Responsible for empirical evaluation, including dataset selection, experimental setup, and parameter tuning for model fine-tuning.
- Yi-Xiang Yu: Leads the problem formulation and methodology design, providing theoretical foundations and ensuring alignment with project goals.

8 Future Work

Surrogate Model-Accelerated Evaluation:
To mitigate the primary computational bottleneck of executing full training runs during hyperparameter evaluation, we propose integrating a multi-dataset surrogate prediction model to estimate task performance given a configuration. While a single-dataset differentiable surrogate allows direct gradient descent optimization, it is fundamentally restricted to single-task scenarios. Within our meta-RL framework, a multi-dataset surrogate functions as a high-fidelity simulation environment. This setup allows the reinforcement learning agent to efficiently learn and transfer sequential search policies across diverse data distributions without the computational overhead of actual model training.

References

Jeroen Albrechts, Hugo M. Martin, and Maryam Tavakol. Model-based meta-reinforcement learning for hyperparameter optimization. In *Intelligent Data Engineering and Automated Learning – IDEAL 2024*, volume 15346 of *Lecture Notes in Computer Science*. Springer, Cham, 2025.

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 2012.

Hadi S. Jomaa, Josif Grabocka, and Lars Schmidt-Thieme. Hyp-rl: Hyperparameter optimization by reinforcement learning. *arXiv preprint arXiv:1906.11527*, 2019.

Kaggle. Kaggle playground series competitions. https://www.kaggle.com/competitions, 2023–2025.

Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, 2012.

Martin Wistuba, Arlind Kadra, and Josif Grabocka. Supervising the multi-fidelity race of hyperparameter configurations. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Jia Wu, Xiyuan Liu, and Senpeng Chen. Hyperparameter optimization through context-based meta-reinforcement learning with task-aware representation. *Knowledge-Based Systems*, 260:110160, 2023.

Lucas Zimmer, Marius Lindauer, and Frank Hutter. Auto-PyTorch tabular: Multi-fidelity metalearning for efficient and robust AutoDL. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(9): 3079–3090, 2021.